# Programming Ex. 6: Regularization

## Machine Learning

Supervised Algorithms

UNIVERSIDAD DE SAN BUENAVENTURA

# Guide #6 – Regularization

## Introduction

The aim of this exercise is to implement a classification algorithm to get a non-linear decision boundary that classifies the dataset in two different groups.

To get started with this exercise, there will be included different files that you have to complete to finish this task. Throughout the exercise, you will execute the script *Lab6.py* which calls functions that you will modify; **please do not modify any other file, just follow the instructions of the assignment.**

## Files included.

Lab6.py – *main script to execute practice #6.*
[*] reading.py – *Function to read the plain file of data.*
[*] logistic.py – *Function to compute the cost and gradient descent algorithm.*
[*] plotting.py – *Function to plot the different dataset figures.*
[*] predict.py – *Function to predict the performance of the model.*
[*] sigmoid.py – *Function to compute the sigmoid function for the hypothesis.*
[*] featuremult.py – *Function to create a polynomial features term.*
* *Indicates files you will need to complete*.

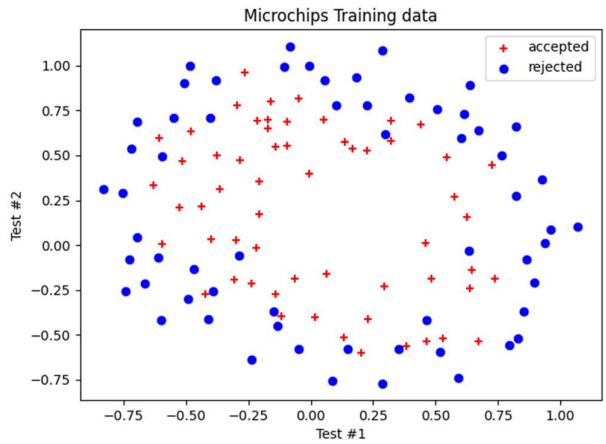
## 1. Data reading function

The file *data2.txt* contains the dataset to implement a logistic regression model. The dataset includes two columns that show the test for different microchips, then the third column indicates whether the microchips were accepted or not. Complete the function ***data( )*** to load the dataset in Python, return *X* with the data of the first two columns and *y* with target dataset. *(same as the previous exercise)*

## 2. Visualizing the dataset

At first, it is a good idea to implement an algorithm to visualize each microchip tests on a 2D graph. It is necessary to differentiate with markers between students who were admitted "+" or not "o".

Once the function ***viewdata( )*** is completed, it will be displayed a figure as follows. *(same as the previous exercise)*




## 3. Feature mapping

An option to fit the data better is to create more features by building a polynomial feature multiplication, noted as *feature mapping.* The idea is to map the features into polynomial terms of x1 and x2 to sixth power, as follows:

$$
\begin{array}{ccccccc}
 & & & 1 & & & \\
 & & x_1 & & x_2 & & \\
 & {x_1}^2 & & x_1x_2 & & {x_2}^2 & \\
\iddots & & \vdots & & \vdots & & \ddots \\
{x_1}^4 & {x_1}^3x_2 & & {x_1}^2{x_2}^2 & & x_1{x_2}^3 & {x_2}^4 \\
\iddots & \vdots & & \vdots & & \vdots & \ddots
\end{array}
$$

As a result, X will pass from 2 features to a matrix of 28 features. This implementation will create a more complex decision boundary, more prone to overfitting by creating a non-linear model when the 2-D plot would be stablished.

In ***mapfeature()***, should be implemented this algorithm. Therefore, to check your algorithm, it is possible to execute ***featuremult.py*** directly to run an unitary test that takes a different example and compares your answer with the correct answer. If your implementation is correct, you will receive on the terminal screen a message as follows:

```
You have passed the test!
```

## 4. Sigmoid Function implementation

To compute the sigmoid function over the dataset, remember that the logistic regression hypothesis is noted as:

$$f_{(\omega,b)} \rightarrow g(z) \rightarrow g(\omega^T x)$$

Where g is the sigmoid function. Complete the function **sigmoid( )**, where for large positives values of x, it returns a value close to 1, while for large negatives values of x, it returns a value close to 0; for sigmoid(0), the return should be 0,5. Remember that your implementation should work for vectors or matrices too.

To test your function, execute the file *sigmoid.py* that already asks for an entry and compute the corresponding function ***sigmoid( )***. *(same as the previous exercise)*

## 5. Cost Function and Gradient Descent

Then to implement the cost function in ***costFunction( )***, it should be computed the hypothesis *f(x)* with the regularization term as the following equation.

$$J_{(\omega)} = -\frac{1}{m}\left[\sum_{i=1}^{m} y_{(i)} \cdot \log\left(f_{(x(i))}\right) + \left(1 - y_{(i)}\right) \cdot \log\left(1 - f_{(x(i))}\right)\right] + \frac{\lambda}{2m}\sum_{j=1}^{n} {\omega_j}^2$$

After the execution of the cost function with the initial parameters specified in the algorithm, the cost function value should be ≈ 0,693.

Likewise, to implement the gradient descent for a logistic regression algorithm, it should be computed the values of ω with the same structure defined in the previous lab and its corresponding regularization term, except for first value of ω (intercept).

$$\omega_0 := \omega_0 - \alpha \frac{1}{m}\sum_{i=1}^{m}\left(f_{(x_{(i)})} - y_{(i)}\right)$$

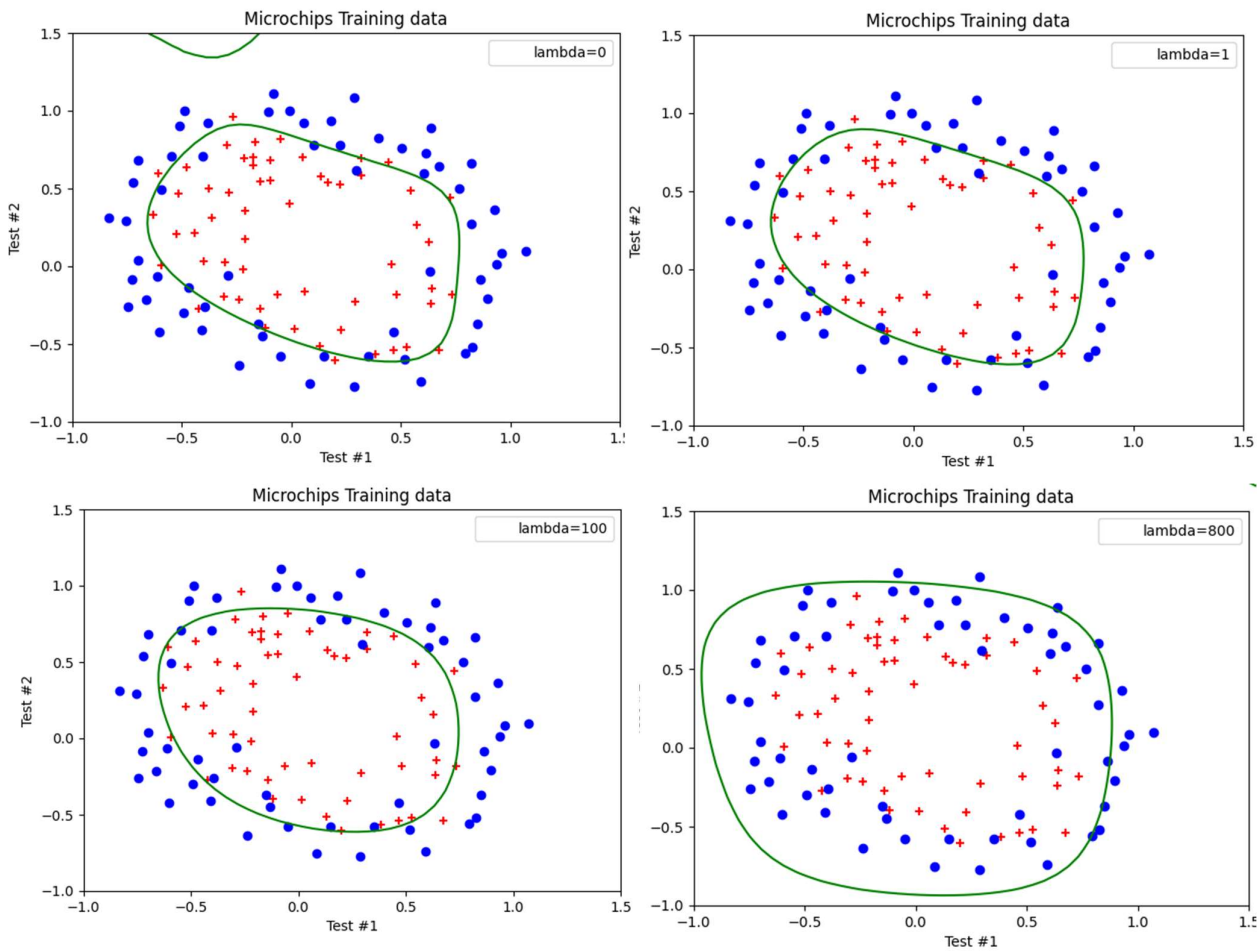
$$\omega_n := \omega_n - \alpha \left[\frac{1}{m}\sum_{i=1}^{m}\left(f_{(x_{(i)})} - y_{(i)}\right) \cdot x_{n(i)} + \frac{\lambda}{m}\,\omega_j\right]$$

Besides to ensure a correct execution of the gradient descent algorithm, it needs a proper *learning rate value* and its corresponding *number of iterations*, just like it was done in a previous programming exercise. It can be used a plot to get the change of the loss over each iteration (*it must always decrease)*; do not change the initial parameters of ω. At first you should find the correct values for ω without regularization, that is, λ=0.

After completing the ***gradientDescent( )*** function, and execute the function without regularization, the cost function value for the optimal value of ω should be ≈ 0.39.

## 6. Plotting the decision boundary

Then you can try different values of λ to see how regularization works in the model.




## 7. Model performance

Finally, complete the function ***prediction( )*** to measure the performance of the model, comparing the estimated output of the model with the actual target of the dataset. Therefore, it should be computed a new vector with the computed values of X according to the model found.

Once the estimated output is calculated, the function returns a vector of ŷ stored in the variable *p* and compare it with the actual target of the dataset *y*, to evaluate how many samples of the dataset are correctly classified according to the known output.

The performance of the model should be > 80%. By changing the regularized term, select the best model and submit your implementation.